

UNIVERSIDADE DA CORUÑA



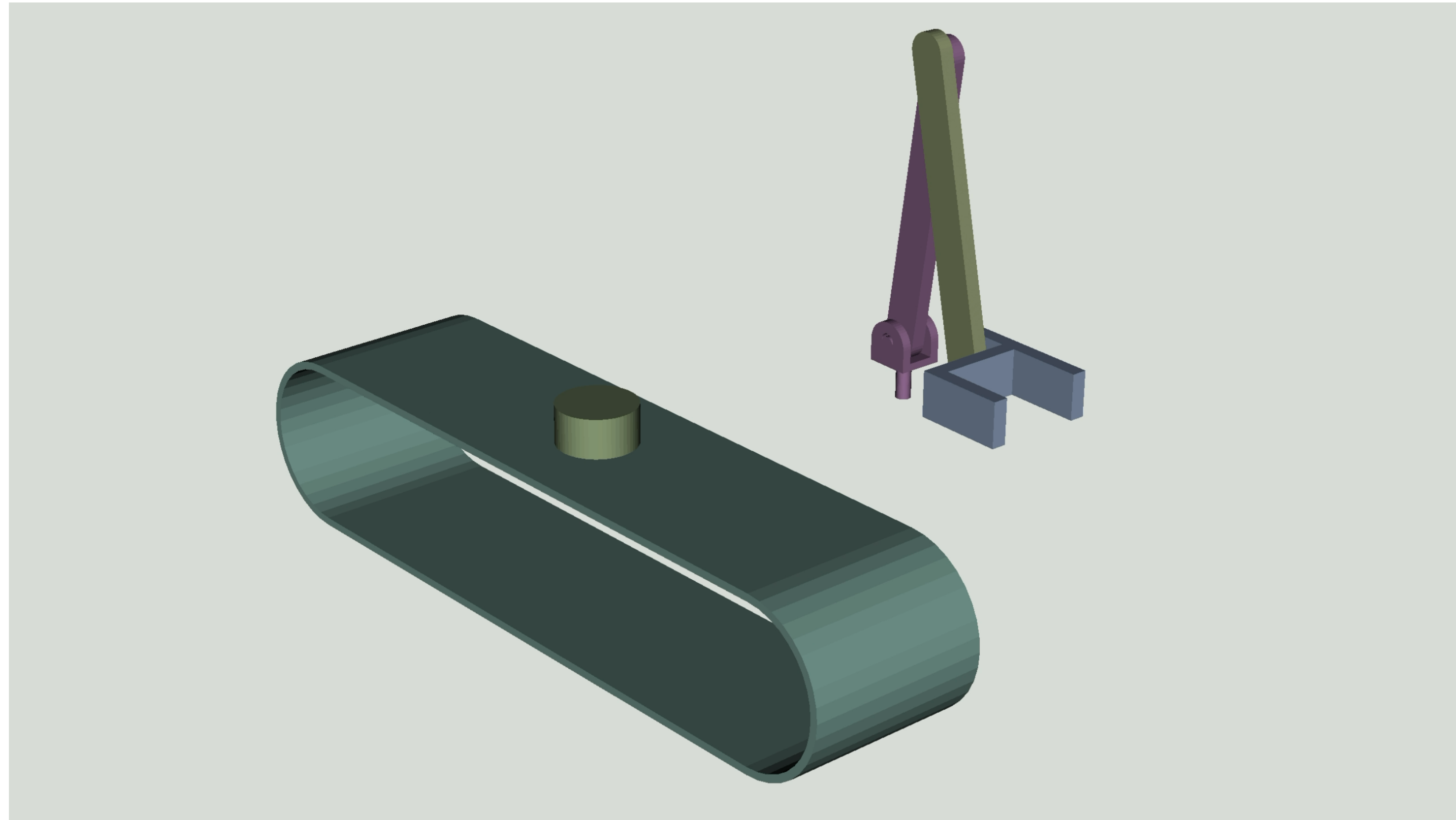
Práctica de programación

Informática

Grados en Ing. Mecánica e Ing. en Tecnologías Industriales - 1^{er} curso

Resumen

- El programa debe generar una *animación* en la cual un brazo robótico recoge latas (envases) sobre una cinta transportadora y las deposita en una zona aparte.

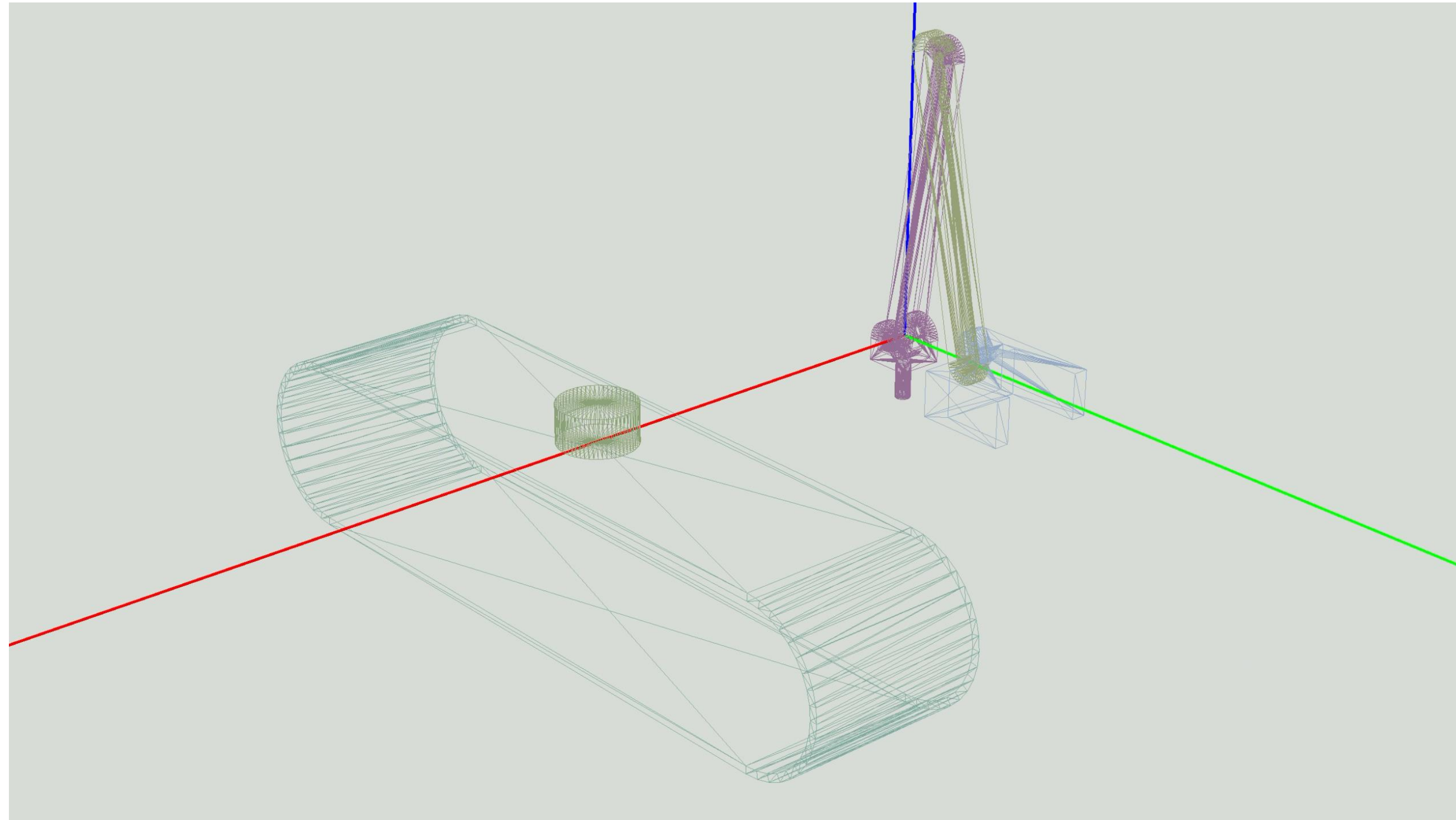


Resumen

- La animación se crea pintando las diferentes piezas que componen la máquina en su posición correcta, que irá cambiando con el paso del tiempo.

Resumen

- Las piezas son mallas compuestas por triángulos, cada uno definido por sus 3 vértices, que tienen unas coordenadas en un sistema de coordenadas cuyo origen está en la base del brazo robótico.



Resumen

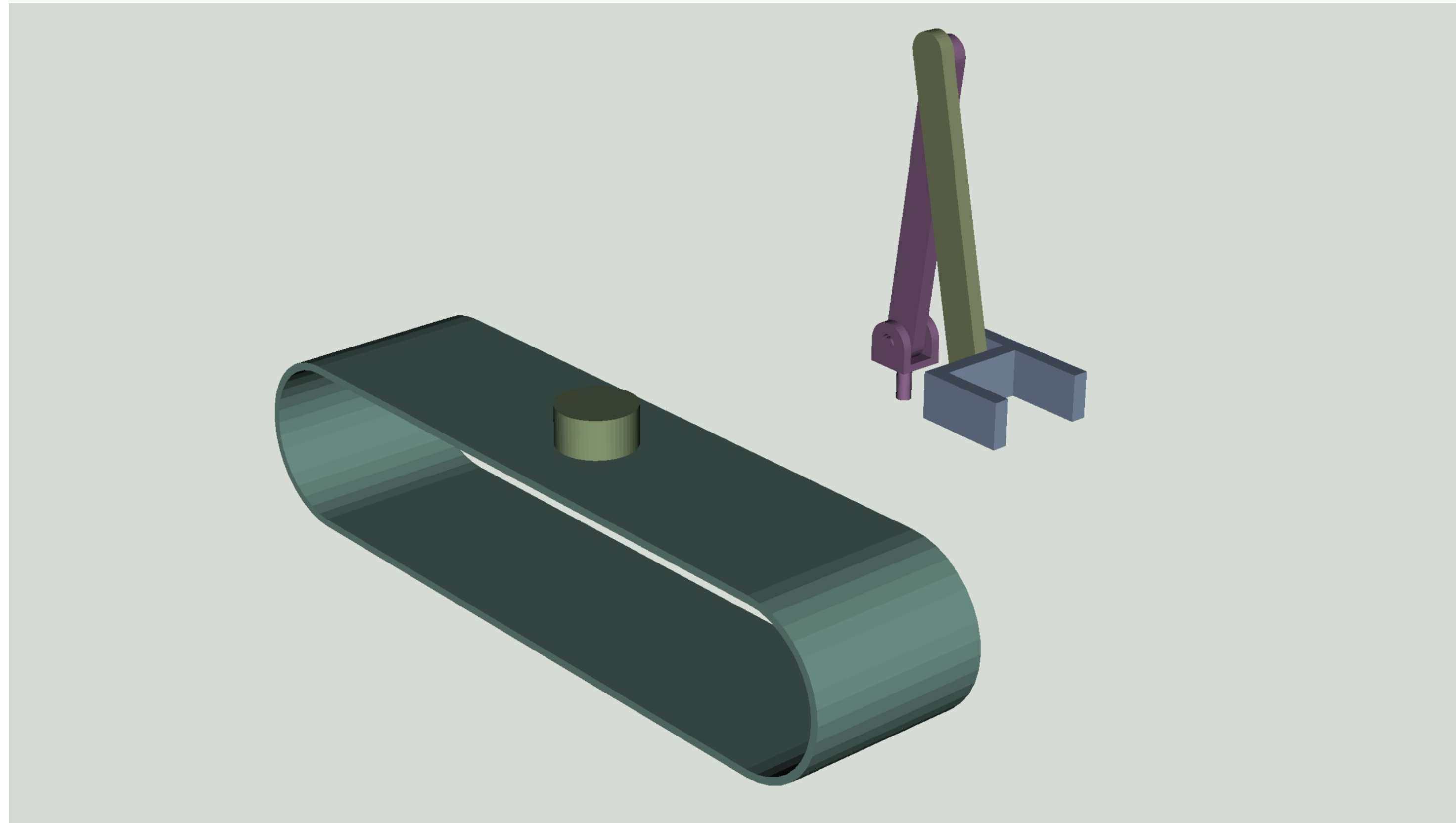
- Para pintar las piezas en su posición correcta en cada instante de tiempo tendremos que ir las rotando y trasladando, cambiando así las coordenadas de los triángulos que las forman, para lo cual usaremos matrices de transformación.

Resumen

- Una posible solución implicaría:
 - Rotar la base, extendiendo los brazos y manteniendo la pinza horizontal, hasta estar cerca de la cinta a la altura deseada y orientado a la lata que se desea agarrar.
 - Desplazarse hacia la lata, extendiendo más los brazos, y agarrarla.
 - Retroceder, encogiendo los brazos y manteniendo la pinza horizontal, hasta estar fuera de la cinta.
 - Rotar la base, extendiendo los brazos y manteniendo la pinza horizontal, hasta poner la lata encima de la zona de descarga.
- Debemos de pintar el brazo en esas *posiciones clave*...
- ... y para crear el efecto deseado de animación, pintarlo también en posiciones intermedias. Cuantas más posiciones intermedias, más fluida será la animación.

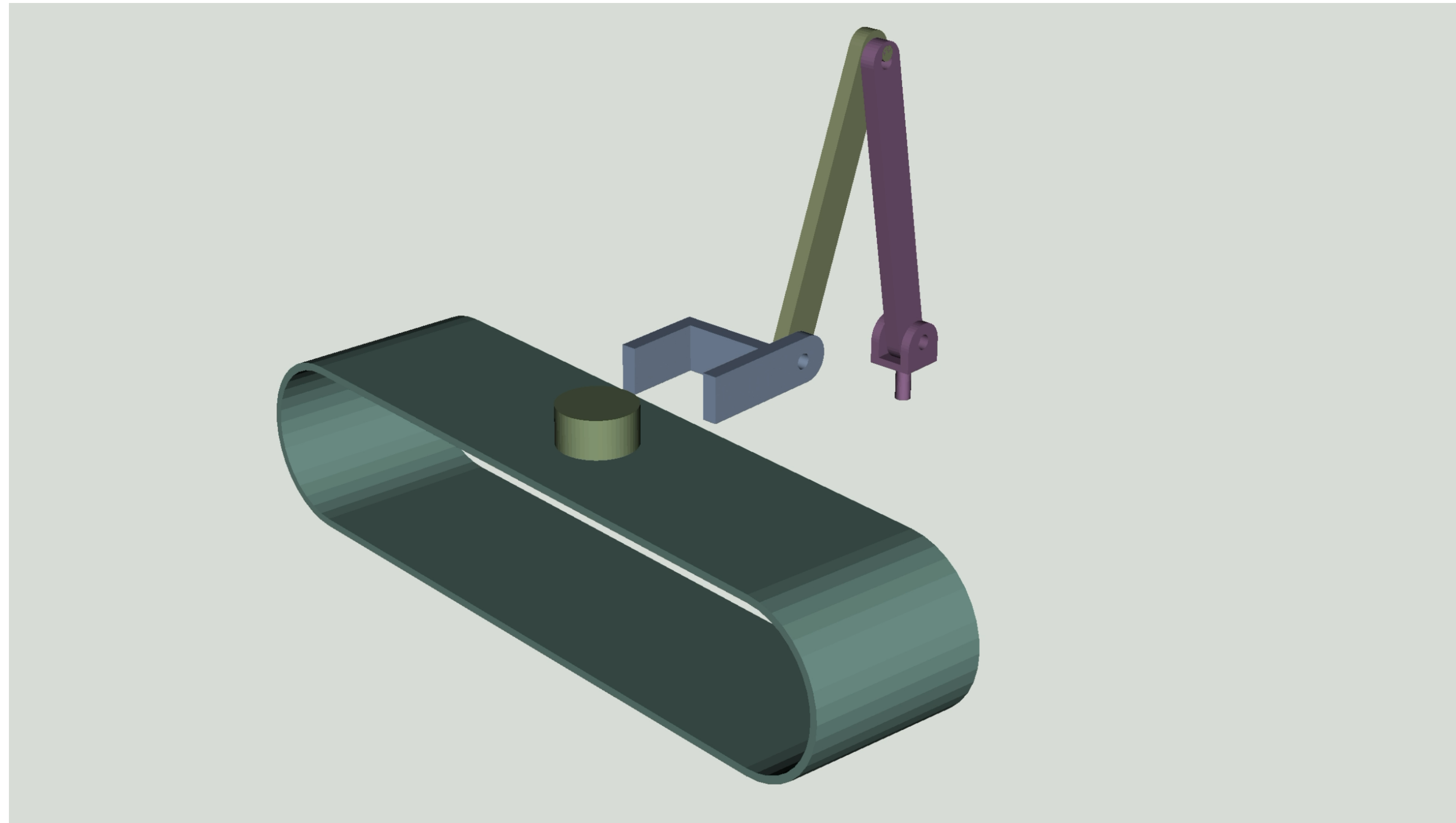
Ejemplo

- Rotar hacia la lata.



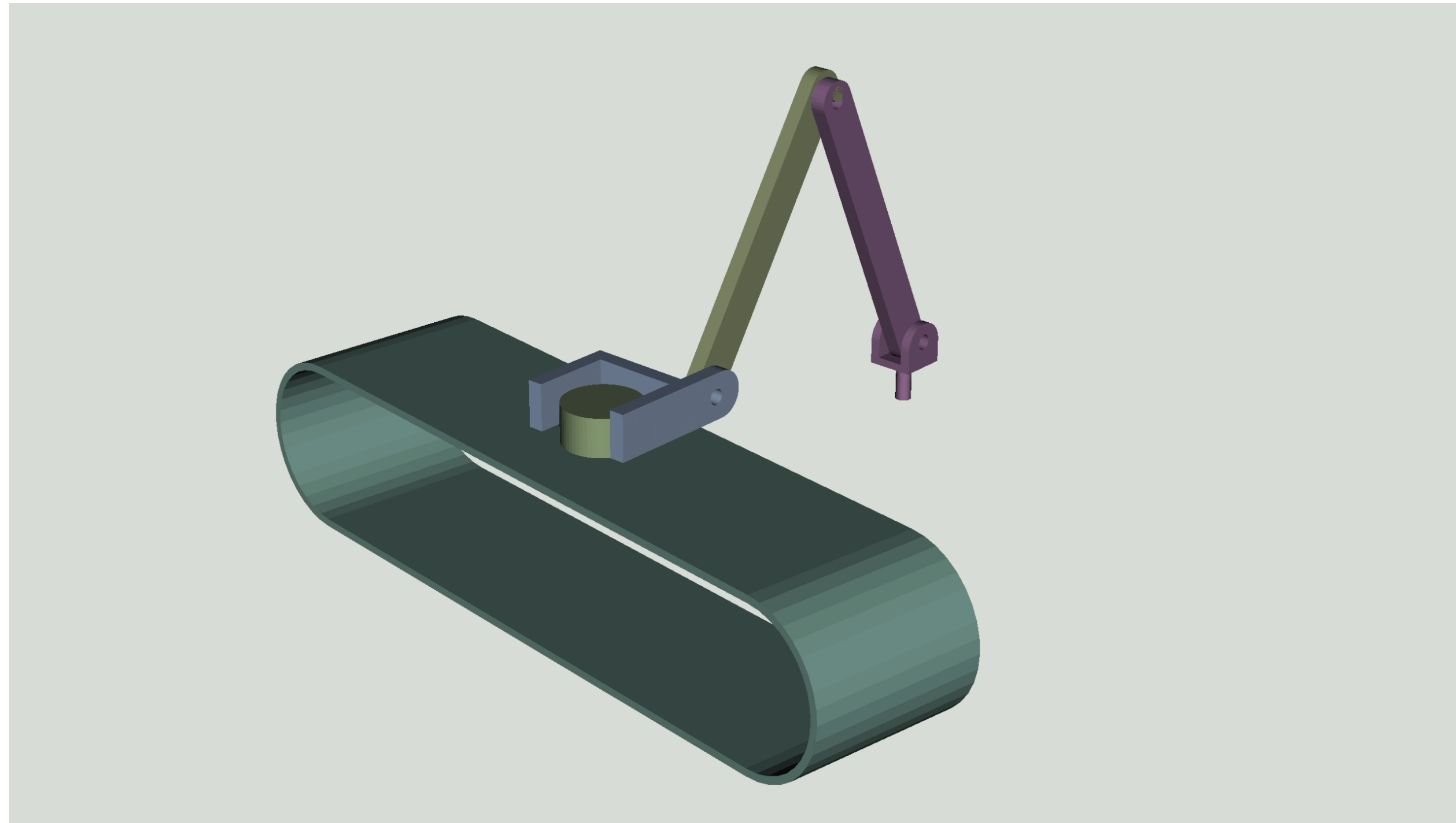
Ejemplo

- Agarrar.



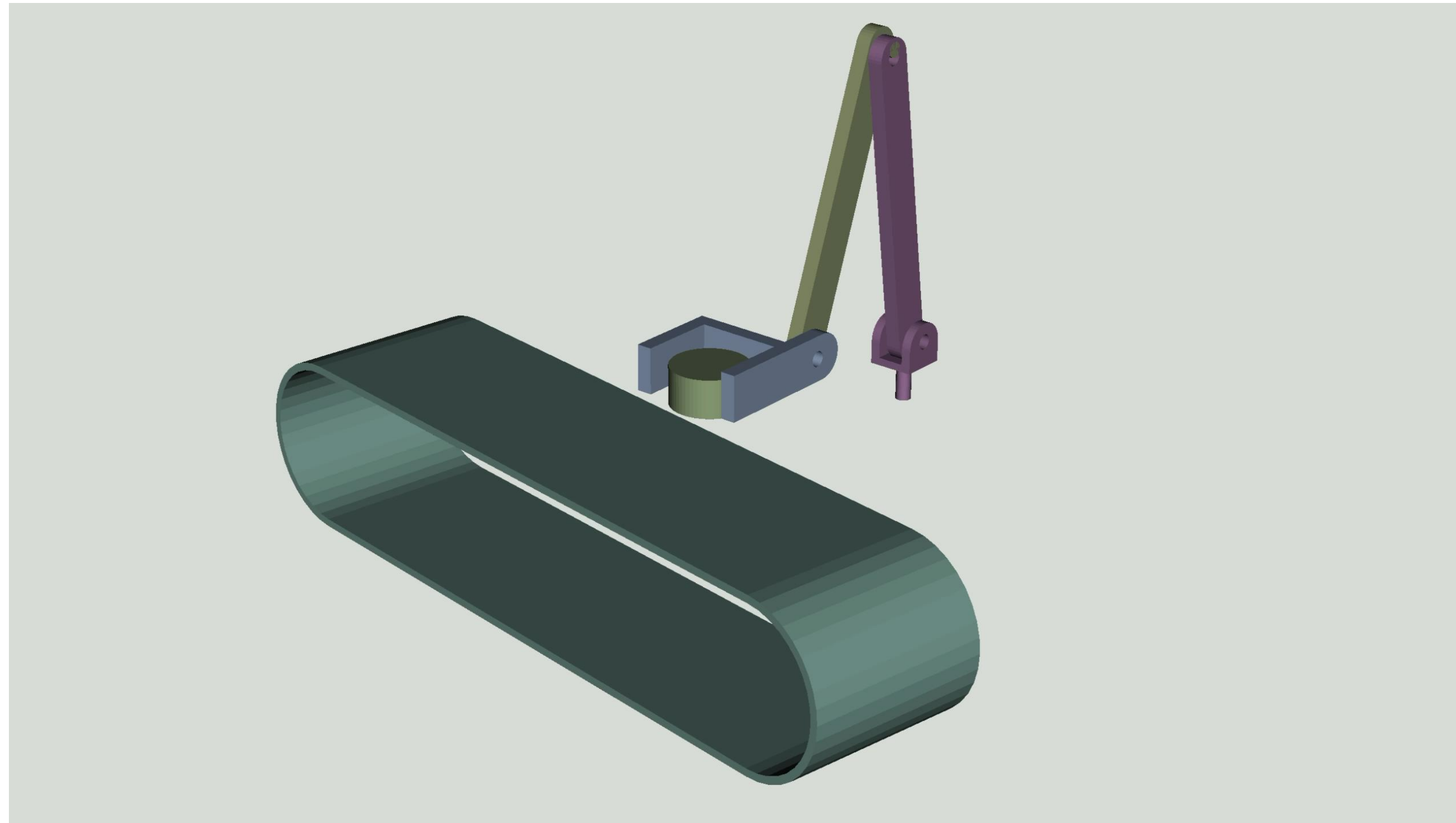
Ejemplo

- Retroceder.



Ejemplo

- Ir a zona de descarga.



Matrices de transformación

MATRICES DE ROTACIÓN EN X, Y, Z

MATRIZ DE TRASLACIÓN

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) & 0 \\ 0 & \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_y(\beta) = \begin{bmatrix} \cos(\beta) & 0 & \sin(\beta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\beta) & 0 & \cos(\beta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_z(\gamma) = \begin{bmatrix} \cos(\gamma) & -\sin(\gamma) & 0 & 0 \\ \sin(\gamma) & \cos(\gamma) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad Tl(v) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

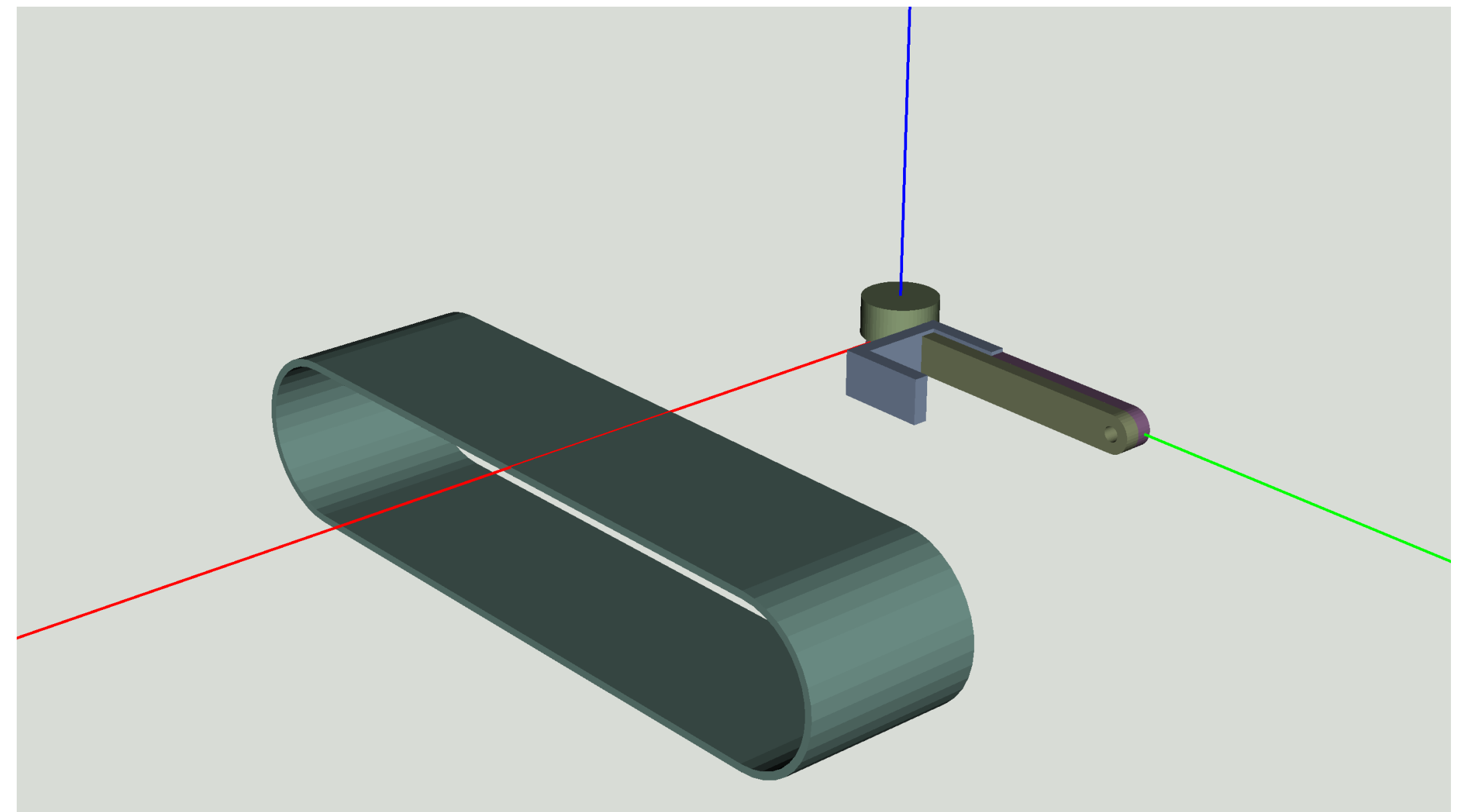
- Una matriz de transformación permite definir una operación de traslación y / o rotación para un punto(s) y se calcula como la multiplicación de las matrices indicadas arriba:
 - $T = Tl @ Rz @ Ry @ Rx$
- Las nuevas coordenadas del punto(s) se pueden calcular mediante la multiplicación de dicha matriz por el vector(es) con las coordenadas del punto(s).
- Utilizaremos la librería `numpy-stl`, que permite aplicar fácilmente una matriz de transformación a una malla (con los puntos que definen un objeto) con el método `transform()`.

Matrices de transformación

- Podemos aplicar varias transformaciones simultáneamente a una pieza multiplicando sus correspondientes matrices de transformación.
- Estas transformaciones debemos de “leerlas” / visualizarlas como si las aplicásemos a la pieza secuencialmente de izquierda a derecha.
- Teniendo en cuenta lo anterior, si queremos mover una pieza (B) solidariamente con otra (A) y, además, que ejecute otro movimiento, solo tenemos que calcular la matriz de transformación de B como el producto de la matriz de transformación de A por otra matriz de transformación que refleje el movimiento adicional que queremos para B.
- A continuación, veremos varios ejemplos sobre esto.

Exportación de las piezas

- Los objetos aparecerán sobre el escenario en las coordenadas que tenían en SolidWorks cuando se exportaron.
- Por ello, se deben de exportar las piezas de forma que el punto sobre el que van a rotar esté en el origen de coordenadas, para que las rotaciones se visualicen correctamente.



Ejemplos

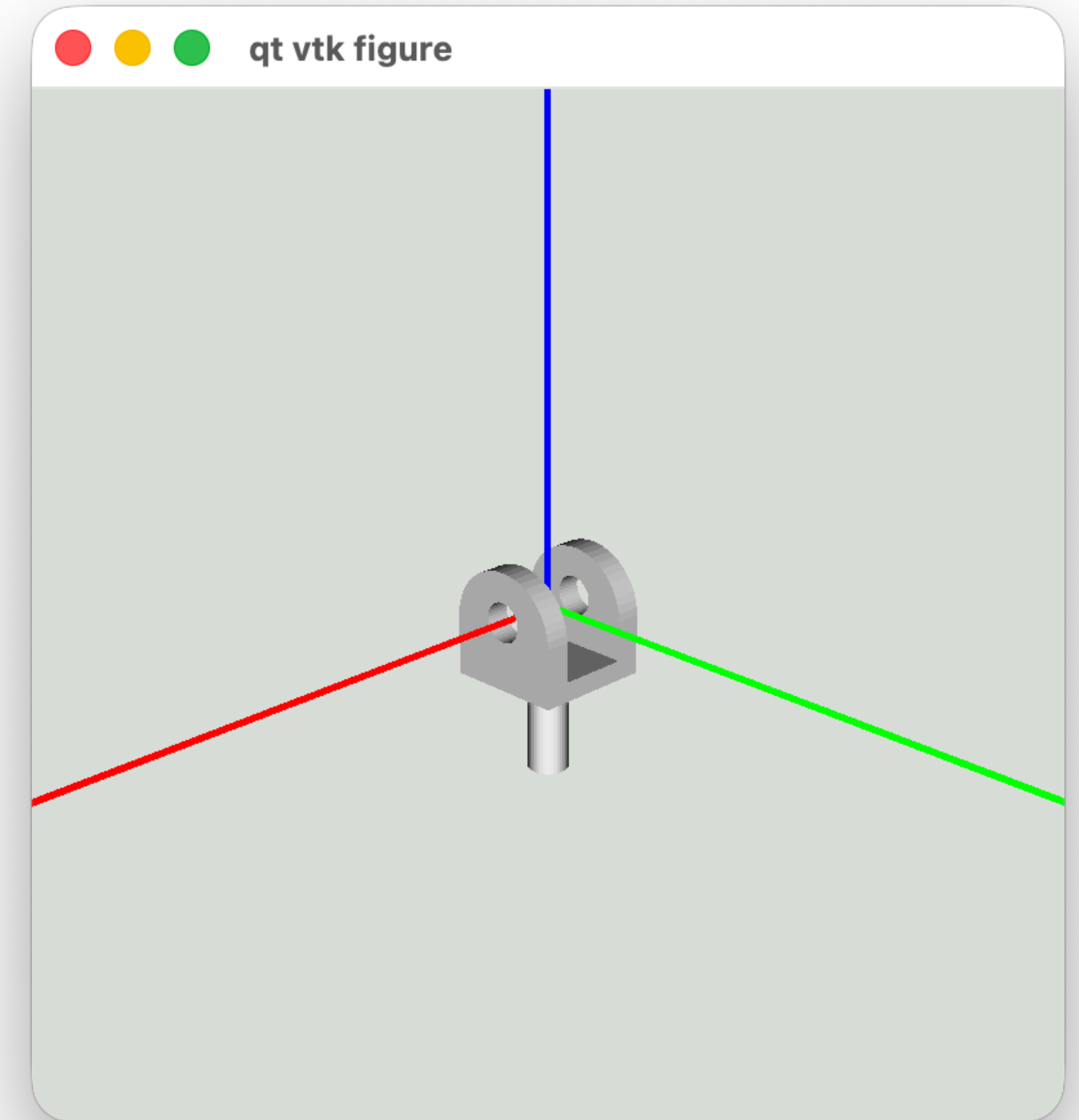
- En los ejemplos que vienen a continuación solo aparecerán algunas de las piezas.
- Estos ejemplos están pensados para probar el código de forma interactiva, pero recordad que lo que se entregue tiene que ser un programa convenientemente estructurado en funciones.

Carga de un modelo

```
import numpy
# Para mostrar la escena.
import vtkplotlib as vpl
# Para cargar los modelos 3D.
from stl import mesh

def pinta_ejes():
    vpl.plot(numpy.array([[0, 0, 0], [3, 0, 0]]), color=(0, 255, 0), line_width=5.0, label="X")
    vpl.plot(numpy.array([[0, 0, 0], [0, 3, 0]]), color=(0, 0, 255), line_width=5.0, label="Y")
    vpl.plot(numpy.array([[0, 0, 0], [0, 0, 3]]), color=(255, 0, 0), line_width=5.0, label="Z")

# Creamos la ventana de visualización.
vpl.QtFigure()
# Cargamos un modelo.
modelo = mesh.Mesh.from_file("base.stl")
# Lo añadimos a la ventana.
vpl.mesh_plot(modelo)
# Pintamos los ejes (solo para ayudar a visualizar)
pinta_ejes()
# Ponemos la cámara donde nos interesa.
vpl.gcf().vl.setContentsMargins(0, 0, 0, 0)
vpl.view(camera_position=(0.6, 0.35, 0.6), focal_point=(0.0, 0.0, 0.0))
# Visualizamos la ventana (la figura se borra al cerrarla).
vpl.show()
```

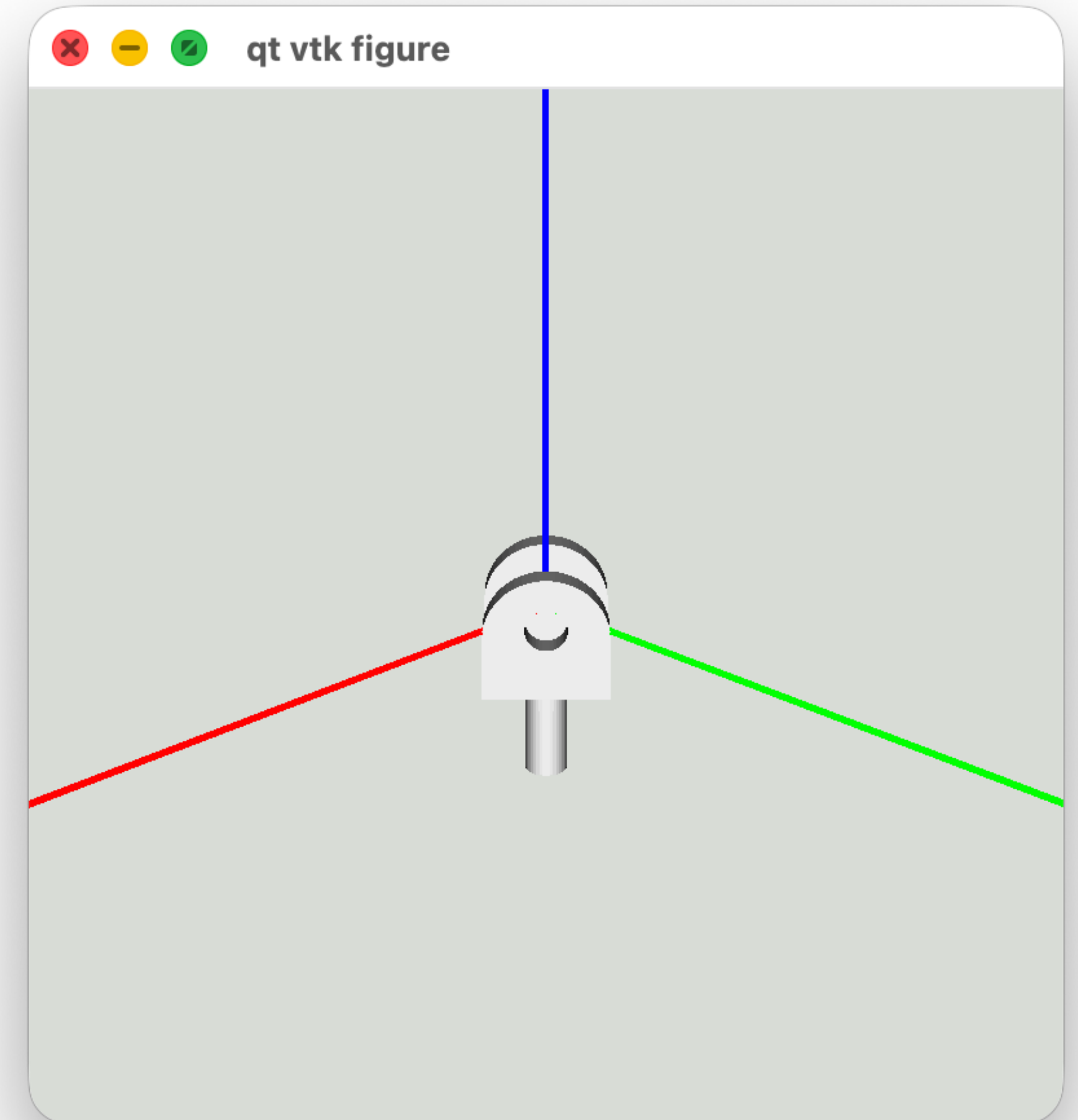


Rotación en Y

```
from copy import deepcopy

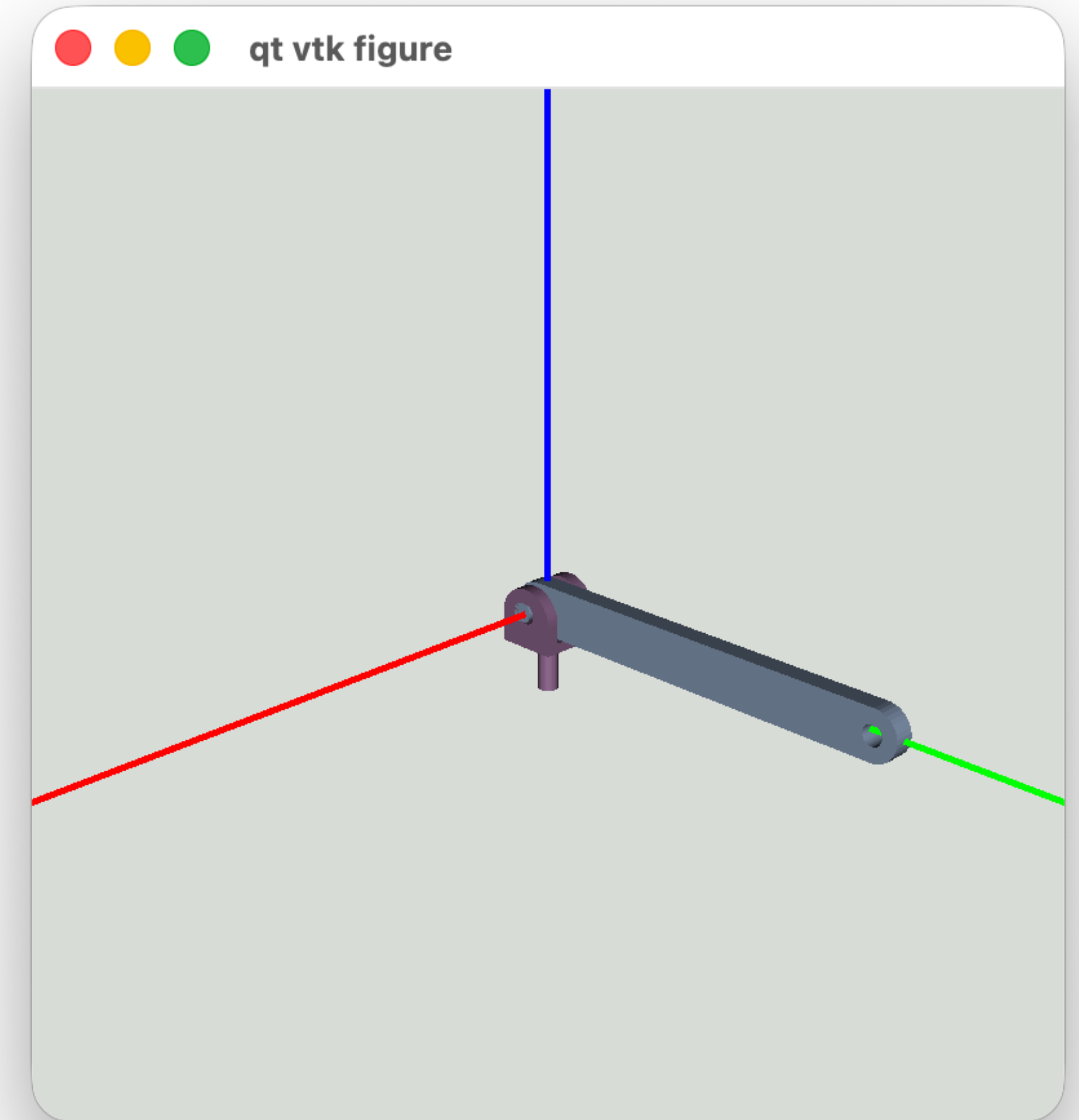
def rotate_y(angle):
    matrix = numpy.identity(4)
    matrix[0, 0:3] = [numpy.cos(angle), 0.0, numpy.sin(angle)]
    matrix[2, 0:3] = [-numpy.sin(angle), 0.0, numpy.cos(angle)]
    return matrix

vpl.QtFigure()
vpl.gcf().vl.setContentsMargins(0, 0, 0, 0)
vpl.view(camera_position=(0.6, 0.35, 0.6), focal_point=(0.0, 0.0, 0.0))
pinta_ejes()
# Calculamos la matriz de transformación para rotar la pieza 45º en Y.
Rx = numpy.identity(4)
Ry = rotate_y(numpy.radians(45))
Rz = numpy.identity(4)
T = numpy.identity(4)
Mt = T @ Rz @ Ry @ Rx
# Copiamos el modelo para no modificar sus coordenadas originales.
copia = deepcopy(modelo)
# Aplicamos la matriz de transformación.
copia.transform(Mt)
vpl.mesh_plot(copia)
vpl.show()
```



Cargar varios modelos

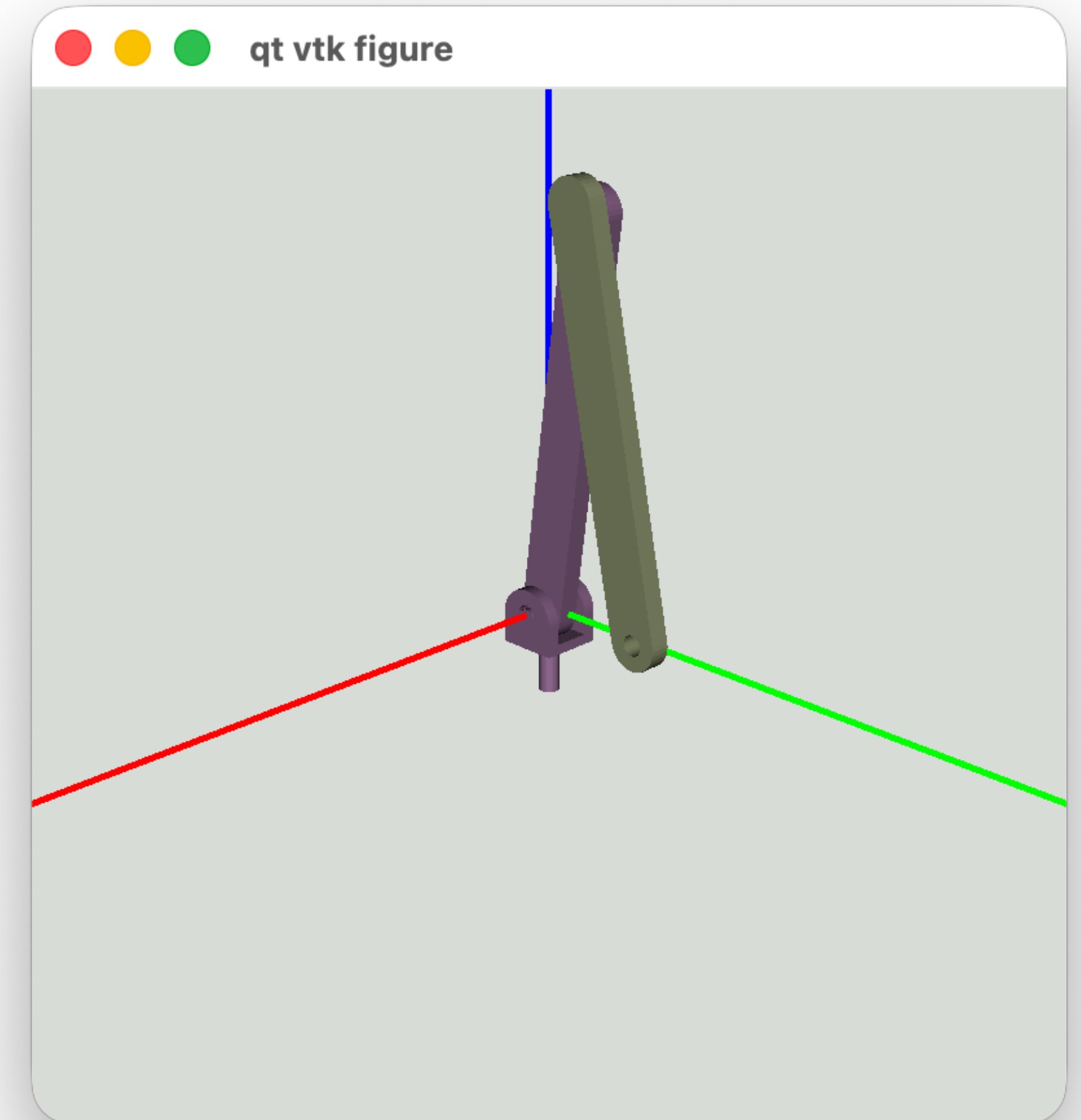
```
vpl.QtFigure()
vpl.gcf().vl.setContentsMargins(0, 0, 0, 0)
vpl.view(camera_position=(1.2, 0.7, 1.2), focal_point=(0.0, 0.0, 0.0))
pinta_ejes()
base = mesh.Mesh.from_file("base.stl")
brazo1 = mesh.Mesh.from_file("brazo1.stl")
brazo2 = mesh.Mesh.from_file("brazo2.stl")
# Esta vez, ponemos colores a las piezas para distinguirlas mejor.
vpl.mesh_plot(base, color=(0.59, 0.43, 0.59))
vpl.mesh_plot(brazo1, color=(0.59, 0.43, 0.59))
vpl.mesh_plot(brazo2, color=(0.59, 0.63, 0.47))
vpl.show()
```



Ensamblar modelos

```
def rotate_z(angle):
    matrix = numpy.identity(4)
    matrix[0:3, 0:3] = [
        [numpy.cos(angle), -numpy.sin(angle), 0.0],
        [numpy.sin(angle), numpy.cos(angle), 0.0],
        [0.0, 0.0, 1.0],
    ]
    return matrix

vpl.QtFigure()
vpl.gcf().vl.setContentsMargins(0, 0, 0, 0)
vpl.view(camera_position=(1.2, 0.7, 1.2), focal_point=(0.0, 0.0, 0.0))
pinta_ejes()
Rz1 = rotate_z(numpy.radians(82.0))
brazo1_copia = deepcopy(brazo1)
brazo1_copia.transform(Rz1)
T = numpy.identity(4)
T[0:3, 3] = [0.39, 0.0, 0.0]
Rz2 = rotate_z(numpy.radians(-160.0))
brazo2_copia = deepcopy(brazo2)
brazo2_copia.transform(Rz1 @ T @ Rz2)
vpl.mesh_plot(base, color=(0.59, 0.43, 0.59))
vpl.mesh_plot(brazo1_copia, color=(0.59, 0.43, 0.59))
vpl.mesh_plot(brazo2_copia, color=(0.59, 0.63, 0.47))
vpl.show()
```



Animación

- En la siguiente diapositiva animamos un movimiento consistente en abrir ambos brazos.
- Ten en cuenta que este ejemplo solo cambia la rotación sobre el eje Z. En otros movimientos puede haber otras traslaciones y / o rotaciones.
- Para este caso particular algunas operaciones podrían simplificarse, pero se han dejado así por claridad.
- La función `paint()`:
 - Añade las piezas a la ventana.
 - Pinta el contenido de la ventana y continua sin esperar a que el usuario la cierre.
 - Elimina las piezas de la ventana.

Animación

```
def paint(maquina):
    lista = []
    for pieza in maquina:
        lista.append(vpl.mesh_plot(pieza))
    figure = vpl.gcf()
    figure.update()
    figure.show(block=False)
    for pieza in lista:
        figure.remove_plot(pieza)

vpl.QtFigure()
vpl.gcf().setWindowState(PyQt6.QtCore.Qt.WindowState.WindowMaximized)
vpl.gcf().vl.setContentsMargins(0, 0, 0, 0)
vpl.view(camera_position=(2.4, 1.4, 2.4), focal_point=(0.0, 0.0, 0.0))
pinta_ejes()
Rx = numpy.identity(4)
Ry = numpy.identity(4)
Rz = numpy.identity(4)
n_steps = 100

for step in range(n_steps):
    ang = step * 40.0 / n_steps
    # Movemos el brazo1
    brazo1_copia = deepcopy(brazo1)
    T = numpy.identity(4)
    Rz = rotate_z(numpy.radians(82.0 - ang))
    Mt_brazo1 = T @ Rz @ Ry @ Rx
    brazo1_copia.transform(Mt_brazo1)
    # Movemos el brazo2
    brazo2_copia = deepcopy(brazo2)
    T = numpy.identity(4)
    T[0:3, 3] = [0.39, 0.0, 0.0]
    Rz = rotate_z(numpy.radians(-160.0 + 2.0 * ang))
    Mt_brazo2 = T @ Rz @ Ry @ Rx
    brazo2_copia.transform(Mt_brazo1 @ Mt_brazo2)
    paint([base, brazo1_copia, brazo2_copia])
```

El incremento en el ángulo de rotación sobre el eje “z” cambia con el número de iteración del bucle:

- Cuando step vale 0 => ang = 0
- Cuando step vale 100 => ang = 40

Animación

